DALIL TRAINING ACADEMY · FULL STACK WEB DEVELOPMENT

Phase 0: دليل القراءة والمهام

مشروع Expense Tracker · يوم 1 – 2 · تعلّم ذاتي

قبل ما تبدأوا

- **الهدف:** تتعلموا 5 مواضيع جديدة خلال يومين، وتتأكدوا إنكم فهمتوها من خلال 3 مهام صغيرة.
- **الـ backend بيتبنى فورًا بعد هالمرحلة (Phase 1، يوم 3).** يعني لازم تخلصوا هالمواضيع كويس قبل ما تبدأوا، لأنه ما رح يكون في وقت تراجعوها فيه وأنتو تبنوا.
- **اقرأوا وجرّبوا:** كل ما تقرأوا مثال، افتحوا VS Code أو الـ Console وشغّلوه. القراءة بدون تجربة ما بتثبّت.
- المصادر بالإنجليزي ومكتوبة ببساطة. مش لازم تفهموا كل كلمة، ركّزوا على الأمثلة والكود.
- ما تقرأوا كل الروابط الجانبية بالصفحات. اقرأوا الجزء المطلوب بس.
- الأوقات تقريبية. إذا احتجتوا أكتر فعادي، بس **إذا علقتوا أكتر من 30 دقيقة اسألوا بالـ Discord.**

توزيع اليومين

اليوم	القراءة	المهام
1	تجهيز البيئة (حسب الـ Roadmap)، ثم المواضيع 1 و 2 و 3	مهمة 1 (fetch + cards)
2	المواضيع 4 و 5	مهمة 2 (Express) ومهمة 3 (Express + PostgreSQL)

المواضيع والمصادر

الموضوع 1: JSON

المصدر	شو تقرأوا منه	وقت تقريبي
MDN: Working with JSON https://developer.mozilla.org/docs/Learn_web_development/Core/Scripting/JSON	شو هو JSON، والفرق بينه وبين الـ JavaScript object، و `JSON.parse` و `JSON.stringify`. هاي الصيغة اللي بيحكي فيها الـ front-end مع الـ backend طول المشروع.	25 دقيقة

بعد القراءة لازم تقدروا تجاوبوا

- شو الفرق بين JSON و JavaScript object؟
- متى بنستخدم `JSON.stringify` ومتى `JSON.parse`؟

الموضوع 2: Promises و async/await و fetch

المصدر	شو تقرأوا منه	وقت تقريبي
javascript.info: Async/await https://javascript.info/async-await	شو يعني `async` و `await`، وكيف نتعامل مع الأخطاء بـ `try/catch`.	30 دقيقة

المصدر	شو تقرأوا منه	وقت تقريبي
MDN: Making network requests with JavaScript https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Scripting/Network_requests	كيف نستخدم `fetch` لجلب بيانات من server وعرضها بالصفحة.	40 دقيقة
MDN: Using Fetch https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch	**مرجع:** ارجعوا له لما تحتاجوا تبعتوا POST مع body و headers. مش لازم تقرأوه كامل.	عند الحاجة

بعد القراءة لازم تقدروا تجاوبوا

- ليش بنحتاج `await` مرتين لما نعمل `fetch` ونقرأ الـ JSON؟
- شو بيصير إذا الـ server رد بـ 404، هل `fetch` بيرمي خطأ لحاله؟ (ابحثوا عن `response.ok`)

الموضوع 3: الـ Backend والـ REST

المصدر	شو تقرأوا منه	وقت تقريبي
MDN: Client-server overview https://developer.mozilla.org/en-US/docs/Learn/Server-side/First_steps/Client-Server_overview	الجزء اللي بيشرح الـ HTTP request والـ response، والـ methods مثل GET و POST.	30 دقيقة
MDN: HTTP response status codes https://developer.mozilla.org/en-US/docs/Web/HTTP/Status	**مرجع:** ركّزوا على 5 أكواد بس: 200 و 201 و 400 و 404 و 500.	15 دقيقة

بعد القراءة لازم تقدروا تجاوبوا

- شو الفرق بين GET و POST و PUT و DELETE؟ وشو بنستخدم كل واحد فيهم؟
- وين ببعتوا البيانات بالـ POST، وشو الـ status code اللي بنرجعه لما نضيف شي جديد؟

الموضوع 4: Node.js و Express و CORS

المصدر	شو تقرأوا منه	وقت تقريبي
Node.js: Introduction https://nodejs.org/en/learn/getting-started/introduction-to-nodejs	شو هو Node.js، وليش بنقدر نشغّل JavaScript خارج المتصفح. اقرأوه بسرعة.	15 دقيقة
Express: Hello world example https://expressjs.com/en/starter/hello-world.html	ابنوا وشغّلوا أول server بنفسكم، خطوة بخطوة.	30 دقيقة
MDN: Cross-Origin Resource Sharing (CORS) https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS	المقدمة فقط: شو هي وليش المتصفح بيمنع بعض الطلبات. ما بدكم تفاصيل الـ preflight.	20 دقيقة

بعد القراءة لازم تقدروا تجاوبوا

- شو الفرق بين تشغيل JavaScript بالمتصفح وبـ Node.js؟
- ليش بعد ما نعدّل كود الـ server لازم نوقفه ونشغّله من جديد؟
- ليش المتصفح ممكن يمنع الـ front-end من الحكي مع server على port ثاني؟

الموضوع 5: Node.js و PostgreSQL

المصدر	شو تقرأوا منه	وقت تقريبي
node-postgres: Queries https://node-postgres.com/features/queries	الاستعلامات بـ parameters ($1 و $2)، وكيف بنوصل للنتيجة بـ result.rows. اقرأوا بانتباه التحذير عن دمج النصوص بالـ SQL.	25 دقيقة
node-postgres: Pool https://node-postgres.com/apis/pool	شو هو الـ Pool وليش بنستخدمه، و pool.query. مش لازم تقرأوا كل الإعدادات.	20 دقيقة

قبل ما تبدأوا: .env هو ملف نصي فيه الإعدادات السرية (مثل كلمة سر الـ database) مفصولة عن الكود. الـ server بيقرأه بحزمة dotenv. ما بنرفعه ولا بنسلّمه أبدًا.

بعد القراءة لازم تقدروا تجاوبوا

- ليش لازم نبعت القيم كـ parameters ($1) بدل ما ندمجها بنص الـ SQL؟
- وين بنلاقي الصفوف اللي رجعتها pool.query؟
- ليش كلمة سر الـ database بتنكتب بملف .env مش بالكود؟

المهام الصغيرة

المهام مصممة إنها تكون نسخ مصغّرة من اللي رح تبنوه بالمشروع. حلّوها بأنفسكم، وإذا علقتوا ارجعوا للمصادر فوق قبل ما تسألوا.

مهمة 1: fetch و Bootstrap cards — يوم 1

الهدف: تجيبوا بيانات حقيقية من API وتعرضوها بالصفحة.

الخطوات

- ابنوا صفحة HTML فيها Bootstrap (CDN).
- اكتبوا دالة async بتجيب https://jsonplaceholder.typicode.com/users بـ fetch.
- لكل مستخدم اعرضوا Bootstrap card فيها: الاسم، الإيميل، المدينة (address.city)، واسم الشركة (company.name).
- الـ cards بتنبني بالـ DOM من البيانات، مش مكتوبة يدويًا بالـ HTML.
- استخدموا try/catch، واعرضوا Bootstrap alert إذا فشل الطلب. جرّبوا تغلطوا بالـ URL عن قصد لتشوفوا الـ alert.
- اعرضوا Bootstrap spinner أثناء التحميل.

خلصت لما...

- ☐ بتظهر 10 بطاقات (cards) فيها البيانات الصح.
- ☐ لما ألخبط بالـ URL بيظهر alert، مش صفحة فاضية.
- ☐ بقدر أشرح ليش استخدمت await مرتين.

مهمة 2: أول Express server — يوم 2

الهدف: تبنوا server صغير بيرجع JSON، وتشوفوا بأعينكم ليش بنحتاج CORS.

الخطوات

- اعملوا مجلد جديد ونفّذوا `npm init -y` وبعدها `npm install express`.
- اعملوا ملف `server.js` فيه route على `GET /api/hello` بيرجع JSON فيه رسالة. استخدموا `res.json`.
- أضيفوا route ثاني `GET /api/expenses` بيرجع array فيها مصروفين ثابتين، بنفس شكل الـ JSON اللي بالـ Roadmap.
- شغّلوا الـ server بـ `node server.js` وافتحوا الرابط من المتصفح، وبعدين جرّبوه من Thunder Client.
- جرّبوا رابط مش موجود وشوفوا شو بيرجع.
- **CORS بالتجربة:** اعملوا صفحة HTML صغيرة وافتحوها بـ Live Server، واعملوا `fetch` لـ `http://localhost:3000/api/expenses`. رح يظهر خطأ CORS بالـ Console.
- بعدين حلّوه: نفّذوا `npm install cors`، وأضيفوا للـ server السطر `app.use(cors())`، وأعيدوا التجربة.

خلصت لما...

- ☐ الـ routes الاثنين بيرجعوا JSON من المتصفح ومن Thunder Client.
- ☐ شفت خطأ CORS بعيني، وبعرف شو الحل.
- ☐ بقدر أشرح بكلماتي شو الـ status code `404`.

مهمة 3: Express و PostgreSQL — يوم 2

الهدف: توصّلوا server Express بـ PostgreSQL وتقرأوا منها، وتشوفوا بأعينكم شو بترجّع مكتبة `pg` فعلًا.

الخطوات

- استخدموا مجلد مهمة 2، ونفّذوا `npm install pg dotenv`.
- أنشئوا database اسمها `expense_tracker_practice` من pgAdmin، واعملوا فيها جدول بسيط (مثلًا جدول `students` بعمودين: `id` و `name`)، وضيفوا صفّين.
- انسخوا `.env.example` (بتلاقوه بالـ starter) لملف جديد اسمه `.env` واكتبوا كلمة سر PostgreSQL.
- بالـ server اكتبوا `require("dotenv").config()`، وأنشئوا `Pool` من مكتبة `pg` بقيم الـ `.env`.
- غيّروا `GET /api/hello` (أو أضيفوا route جديد) لترجع الصفوف من الجدول: `pool.query("SELECT * FROM students ORDER BY id")` وبعدها `result.rows`. الدالة لازم تكون `async`.
- أضيفوا route فيه parameter: `pool.query("SELECT * FROM students WHERE id = $1", [id])`. إذا ما لقيتوا صف ارجعوا 404.
- جرّبوا الرابط بـ id نص مش رقم (مثل `/abc`). شو صار؟ PostgreSQL ما بتقبل نص مكان رقم. حلّوها بإنكم تتأكدوا من الـ id قبل الاستعلام وترجعوا 404.

خلصت لما...

- ☐ الـ route بيرجع الصفوف من الـ database، مش من array مكتوب بالكود.
- ☐ الرابط بـ id غلط بيرجع 404، مش خطأ بالـ server.
- ☐ بقدر أشرح ليش استخدمنا `$1` بدل ما نكتب الـ id جوا نص الـ SQL.

أخطاء متوقعة

هدول الأخطاء اللي أغلب الطلاب بيقابلوها. اقرأوا الجدول قبل ما تسألوا.

الرسالة	غالبًا معناها
`Cannot GET /something`	الـ route مش موجود أو مكتوب غلط. وممكن تكونوا عدّلتم الكود وما أعدتوا تشغيل الـ server.
`blocked by CORS policy`	الـ server ما سمح للصفحة تحكي معه. استخدموا حزمة `cors`.
`Unexpected token < ... is not valid JSON`	الرد اللي وصلكم كان صفحة HTML مش JSON، غالبًا الـ URL غلط.
`Failed to fetch`	الـ server مش شغال أو الـ port غلط. وممكن يظهر معها `ERR_CONNECTION_REFUSED`.
`EADDRINUSE: address already in use`	في server ثاني شغال على نفس الـ port. أوقفوه بـ Ctrl+C أو غيّروا الـ port.
`await is only valid in async functions`	استخدمتوا `await` جوا دالة مش `async`. أضيفوا `async` قبل الدالة.
`password authentication failed for user`	كلمة السر بملف `.env` غلط. تأكدوا منها بـ pgAdmin.
`database "..." does not exist`	ما أنشأتوا الـ database، أو اسمها بملف `.env` غلط.
`relation "..." does not exist`	الجدول مش موجود بالـ database اللي وصلتوا فيها.
`ECONNREFUSED 127.0.0.1:5432`	PostgreSQL مش شغّال، أو الـ port بملف `.env` غلط.
`invalid input syntax for type integer`	بعتوا نص مكان رقم للـ database. تأكدوا من الـ id قبل الاستعلام.

لما تخلصوا

راجعوا الـ Checklist الخاصة بـ Phase 0 بالـ Roadmap، وسلّموا المجلد اللي فيه المهام الثلاث نهاية يوم 2. بعدها بتبدأوا Phase 1: بناء الـ backend كامل.